

Memoria del Análisis Nextflow

Pipeline de RNA-seq en Nextflow (Dataset GSE52778)

Miguel Ángel Vicente Mesonero
Gloria Muñoz Martín
Javier Luque Serrano

9 de junio de 2026

Resumen

Este documento describe el desarrollo, la ejecución y los resultados de un pipeline bioinformático completo para el análisis de datos de RNA-seq del dataset GSE52778. El pipeline fue construido en Nextflow DSL2, integrando contenedores Docker y ejecutado en el HPC Picasso mediante Singularity. A lo largo del proceso, cada herramienta fue encapsulada como módulo independiente, desarrollada y testeada en ramas separadas de GitHub antes de su integración en el flujo principal. Finalmente, se detallan los problemas técnicos encontrados, especialmente en la integración del análisis exploratorio en R, y las soluciones adoptadas.

Índice

1. Introducción y Descripción del Pipeline	3
2. Desarrollo del Pipeline: Metodología de Trabajo	3
2.1. Estrategia de Desarrollo con GitHub	3
2.2. Integración en el Workflow Principal	3
3. Ejecución en el HPC Picasso: Perfil Singularity	4
3.1. Configuración del Entorno HPC	4
3.2. Ejecución del Pipeline	4
3.3. Consideraciones de Recursos	5
4. Problema Técnico: Integración del Análisis Exploratorio en R	5
4.1. Descripción del Problema	5
4.2. Solución Adoptada: Ejecución Independiente	5
5. Módulos y Parámetros Utilizados	6
5.1. FastQC	6
5.2. Trimmomatic	6
5.3. STAR_INDEX	6
5.4. STAR_ALIGN	6
5.5. SAMtools	7
5.6. Picard MarkDuplicates	7
5.7. FeatureCounts	7
5.8. MultiQC	7
5.9. Módulo de R (Análisis Exploratorio)	8
6. Resultados y Estructura de Salidas	8

7. Interpretación de los Resultados	8
7.1. Interpretación del Control de Calidad (QC)	8
7.2. Interpretación del Análisis Exploratorio (R)	9
8. Instrucciones de Reproducción	9
8.1. Requisitos Previos	9
8.2. Ejecución Local (Docker)	10
8.3. Ejecución en HPC Picasso (Singularity)	10
8.3.1. Contenido del script de lanzamiento (script.sh)	10
8.4. Ejecución del Análisis en R (Workaround)	11

1. Introducción y Descripción del Pipeline

El presente informe recoge la memoria del análisis del pipeline de RNA-seq desarrollado como parte del Módulo 8 de Bioinformática Aplicada del Máster en Bioinformática de la Universidad Europea de Madrid.

El objetivo era construir un pipeline bioinformático reproducible, portable y escalable que integrase contenedores Docker y Nextflow, aplicado a un análisis real de RNA-seq. El dataset seleccionado fue el **GSE52778**, que contiene datos de secuenciación de células musculares lisas de la vía aérea humana (HASM) de 4 donantes masculinos sanos. Cada donante fue tratado con dexametasona $1\mu\text{M}$ durante 18 horas, generando un diseño pareado de $4 \times 2 = 8$ muestras (4 no tratadas y 4 tratadas).

El pipeline está diseñado para ejecutarse desde el control de calidad inicial hasta el análisis exploratorio estadístico, con un único comando. Su estructura sigue las mejores prácticas de Nextflow DSL2, donde cada herramienta está encapsulada como un módulo independiente y la ejecución se orquesta desde el archivo principal `main.nf`.

2. Desarrollo del Pipeline: Metodología de Trabajo

2.1. Estrategia de Desarrollo con GitHub

Durante el desarrollo del pipeline, se adoptó una metodología de trabajo modular y controlada mediante el sistema de control de versiones Git y la plataforma GitHub. **Cada módulo del pipeline se escribió y testeó de manera independiente utilizando distintas ramas de GitHub** (e.g., `feature/fastqc`, `feature/trimmomatic`, `feature/star-align`).

Esta estrategia permitió:

- Aislar el desarrollo de cada herramienta bioinformática, evitando que los errores en un módulo afectaran a los demás.
- Realizar pruebas unitarias de cada proceso de Nextflow (**process**) de forma independiente, ejecutándolo con datos de prueba.
- Facilitar la revisión de código y la integración progresiva mediante *Pull Requests*.
- Mantener un historial limpio y rastreable de las decisiones técnicas y los cambios realizados.

Una vez validado cada módulo en su rama correspondiente, se procedió a la integración en la rama principal (`main` o `master`), conectando todos los procesos en el workflow global de `main.nf`.

2.2. Integración en el Workflow Principal

El workflow principal `main.nf` orquesta la ejecución de los siguientes procesos en orden:

1. **FastQC**: Control de calidad de las lecturas crudas.
2. **Trimmomatic**: Eliminación de adaptadores y lecturas de baja calidad.
3. **FastQC (post-trim)**: Control de calidad de las lecturas limpias.
4. **STAR_INDEX**: Generación del índice del genoma humano (GRCh38) de una sola vez.
5. **STAR_ALIGN**: Alineamiento de cada muestra al genoma de referencia (8 ejecuciones en paralelo).
6. **SAMtools**: Ordenación, indexado y cálculo de métricas de los archivos BAM.
7. **Picard MarkDuplicates**: Marcado de duplicados de PCR sin eliminarlos.

8. **featureCounts**: Cuantificación de las lecturas por gen, utilizando todos los BAMs simultáneamente.
9. **MultiQC**: Agregación de todos los informes de calidad en un único archivo HTML.

Cada proceso se implementa en un archivo separado dentro de `scripts/modules/`, cumpliendo con los requisitos del DSL2: `tag`, `container` (con etiqueta exacta), `publishDir`, `input` y `output` con `emit` nombrado.

3. Ejecución en el HPC Picasso: Perfil Singularity

3.1. Configuración del Entorno HPC

El pipeline completo fue ejecutado en el clúster de alto rendimiento (HPC) Picasso, perteneciente al SCBI-UMA. Este clúster utiliza **Singularity**/**Apptainer** en lugar de Docker para la gestión de contenedores, y **Slurm** como gestor de colas de trabajos.

Para adaptar el pipeline a este entorno, se creó un perfil de ejecución específico denominado **picasso** dentro del archivo `nextflow.config`. La configuración mínima incluye:

```
profiles {
  docker {
    docker.enabled = true
  }
  picasso {
    singularity.enabled    = true
    singularity.autoMounts = true
    singularity.cacheDir =
      "${System.getenv('HOME')}/.singularity_cache"
  }
}
```

Esta configuración permite que Nextflow descargue las imágenes Docker definidas en los procesos, las convierta automáticamente a formato `.sif` (Singularity Image Format) y las ejecute de forma transparente.

3.2. Ejecución del Pipeline

El pipeline se lanzó en Picasso utilizando el perfil **picasso** con el siguiente comando:

```
nextflow run scripts/main.nf -profile picasso
```

Para reanudar una ejecución interrumpida (aprovechando el sistema de caché de Nextflow), se utilizó:

```
nextflow run scripts/main.nf -profile picasso -resume
```

Se debe destacar que la conversión de las imágenes Docker a Singularity es un proceso lento y que consume una cantidad significativa de espacio en disco. Por ello, se configuró el directorio de caché en `$HOME` para evitar problemas de cuota en el sistema de archivos temporal `$FSCRATCH`. La caché de Singularity **no debe eliminarse** entre ejecuciones, ya que es la clave para la reanudación eficiente del pipeline.

3.3. Consideraciones de Recursos

En Picasso, el paso de **STAR_INDEX** es el más exigente en términos de recursos, requiriendo al menos **64 GB de RAM** y **12 CPUs** para la indexación del genoma humano completo (GRCh38). En el perfil de configuración, se asignaron los siguientes recursos:

- **STAR_INDEX** y **STAR_ALIGN**: 12 CPUs, 64 GB RAM.
- **Procesos por defecto**: 2 CPUs, 4 GB RAM.

La ejecución local de todo el pipeline para las 8 muestras tiene un tiempo aproximado de 1.5 horas. En el entorno HPC, el tiempo total depende del estado de la cola de Slurm, siendo el indexado el paso que más tiempo consume. El tiempo final de ejecución dependerá de la disponibilidad de recursos en el clúster, que en este caso fue de 01:57:41.

4. Problema Técnico: Integración del Análisis Exploratorio en R

4.1. Descripción del Problema

Una de las etapas finales del pipeline consiste en el análisis exploratorio estadístico de la matriz de expentas obtenida, utilizando R. El objetivo es producir, como mínimo, un *boxplot* de la distribución de expresión antes y después de la normalización y un Análisis de Componentes Principales (PCA) no supervisado.

El script de análisis, `scripts/bin/rnaseq_exploratory.R`, fue desarrollado para utilizar las librerías `edgeR`, `ggplot2`, `dplyr` y `tidyr`. Inicialmente, se intentó integrar este paso dentro del workflow de Nextflow mediante un módulo específico (`09_rnaseq_r.nf`) que utilizaba la imagen de contenedor `bioconductor/bioconductor_docker:RELEASE_3_18`.

Sin embargo, durante la ejecución en el clúster Picasso, surgieron problemas severos de dependencias con las librerías de R dentro del contenedor. Las imágenes base de Bioconductor, aunque diseñadas para este propósito, presentaron inconsistencias en la instalación de las dependencias del sistema (como `gfortran`, `libcurl` o `libxml2`) que impedían la compilación e instalación de los paquetes de R necesarios (especialmente `edgeR` y sus dependencias de C/C++). Los intentos de resolver estas dependencias mediante instrucciones de pre-instalación en el script o mediante la construcción de una imagen Docker personalizada se descartaron por la complejidad que añadían al pipeline y por el riesgo de incumplir la rúbrica de reproducibilidad.

4.2. Solución Adoptada: Ejecución Independiente

Frente a esta situación, y para no detener el flujo de trabajo principal, se tomó la decisión pragmática de **ejecutar el script `rnaseq_exploratory.R` de manera independiente**, utilizando la versión nativa de R instalada en el sistema del clúster Picasso.

El clúster Picasso cuenta con un módulo de R cargable (e.g., `module load R/4.3.3`) que incluye las librerías necesarias o permite su instalación local en el directorio del usuario sin conflictos con el sistema. El script se ejecutó fuera de la orquestación de Nextflow, directamente en el nodo de login o en un nodo de trabajo interactivo, de la siguiente manera:

```
module load R/4.3.3
Rscript scripts/bin/rnaseq_exploratory.R
```

Esta solución permitió obtener los resultados del análisis exploratorio (gráficos de boxplot y PCA) sin comprometer la ejecución del pipeline principal. Es una decisión documentada que refleja un problema real de compatibilidad en entornos de HPC con contenedores de Bioconductor, y demuestra la capacidad de adaptación ante contingencias técnicas.

5. Módulos y Parámetros Utilizados

Todos los procesos del pipeline utilizan imágenes de contenedor con etiquetas (tags) exactas, incluyendo el hash de build de Bioconda cuando corresponde, tal como se requiere en la rúbrica. A continuación, se detalla cada paso.

5.1. FastQC

- **Función:** Control de calidad de las lecturas de secuenciación (crudas y post-trimming).
- **Imagen:** `quay.io/biocontainers/fastqc:0.12.1-hdfd78af_0`
- **Salidas:** Archivos `.html` y `.zip` que se integran en el informe MultiQC.

5.2. Trimmomatic

- **Función:** Eliminación de secuencias de adaptadores Illumina y recorte de bases de baja calidad.
- **Imagen:** `quay.io/biocontainers/trimmomatic:0.39-hdfd78af_2`
- **Parámetros:**
 - `phred33`: Codificación de calidad en formato Phred+33.
 - `ILLUMINACLIP`: Detección y eliminación de adaptadores.
 - `LEADING:3`: Eliminación de bases iniciales de calidad inferior a 3.
 - `TRAILING:3`: Eliminación de bases finales de calidad inferior a 3.
 - `SLIDINGWINDOW:4:15`: Corte de lecturas cuando la media de calidad de una ventana de 4 bases cae por debajo de 15.
 - `MINLEN:36`: Descarte de lecturas menores a 36 bases tras el recorte.

5.3. STAR_INDEX

- **Función:** Generación del índice del genoma de referencia humano (GRCh38) a partir del FASTA y el GTF (Ensembl release 111).
- **Imagen:** `quay.io/biocontainers/star:2.7.11b-h5ca1c30_8`
- **Parámetros:**
 - `sjdbOverhang 62`: El valor óptimo es la longitud de las lecturas menos 1. En este caso, las lecturas son de 63 bp, por lo que se utiliza 62.
 - `genomeSAsparseD 1`: Reduce el uso de RAM a costa de una ligera disminución de la velocidad, útil para el indexado en entornos con recursos limitados.

5.4. STAR_ALIGN

- **Función:** Alineamiento de las lecturas FASTQ al genoma de referencia indexado.
- **Imagen:** `quay.io/biocontainers/star:2.7.11b-h5ca1c30_8`
- **Parámetros:**
 - `outSAMtype BAM SortedByCoordinate`: El output se genera directamente como un archivo BAM ordenado por coordenadas genómicas, optimizando el paso siguiente.

5.5. SAMtools

- **Función:** Procesamiento de los archivos BAM generados por STAR.
- **Imagen:** `quay.io/biocontainers/samtools:1.19.2-h50ea8bc_1`
- **Acciones:**
 - `sort`: Ordenación del BAM por coordenadas.
 - `index`: Creación del índice `.bai`.
 - `flagstat`: Estadísticas de alineamiento (mapeados, no mapeados, paired, etc.).
 - `idxstats`: Estadísticas de lecturas por cromosoma.
- Las salidas `flagstat` e `idxstats` se pasan a MultiQC.

5.6. Picard MarkDuplicates

- **Función:** Marcado de duplicados ópticos y de PCR **sin eliminarlos**.
- **Imagen:** `quay.io/biocontainers/picard:3.1.1-hdfd78af_0`
- **Parámetros:**
 - `REMOVE_DUPLICATES=false`: Mantiene todas las lecturas originales.
 - `ASSUME_SORT_ORDER=coordinate`: Asume que el BAM de entrada está ordenado por coordenadas.
 - `VALIDATION_STRINGENCY=LENIENT`: Relaja la validación de formato para evitar errores por inconsistencias menores.
- **Nota:** El parámetro `-Xmx` de la JVM se ajusta al 80 % de la RAM asignada al proceso por Nextflow.
- Las métricas de duplicados se envían a MultiQC.

5.7. FeatureCounts

- **Función:** Cuantificación de las lecturas por gen. Utiliza todos los BAMs con duplicados marcados como entrada conjunta.
- **Imagen:** `quay.io/biocontainers/subread:2.1.1-h577a1d6_0`
- **Salidas:**
 - `featurecounts.txt`: Matriz de cuentas con genes en filas y muestras en columnas.
 - `featurecounts.txt.summary`: Resumen de asignación de lecturas.
- El resumen se integra en el informe MultiQC.

5.8. MultiQC

- **Función:** Agregación de todos los informes de calidad en un único informe HTML interactivo.
- **Imagen:** `quay.io/biocontainers/multiqc:1.35-pyhdfd78af_1`
- **Fuentes de QC integradas:**

- FastQC (raw y post-trim)
- Trimmomatic logs
- STAR Log.final.out
- SAMtools flagstat
- Picard MarkDuplicates metrics
- FeatureCounts summary

5.9. Módulo de R (Análisis Exploratorio)

- **Función:** Generación de gráficos de exploración de datos (boxplots y PCA).
- **Script:** `scripts/bin/rnaseq_exploratory.R`
- **Estado:** Este paso **no se integró en la ejecución automatizada de Nextflow** debido a los problemas de dependencias de las librerías de R dentro del contenedor de Bioconductor en el entorno Picasso. Se ejecutó de forma manual usando la instalación nativa de R del clúster.

6. Resultados y Estructura de Salidas

Tras la ejecución exitosa del pipeline principal, los resultados se organizan en el directorio `results/` con la siguiente estructura:

```
results/
| fastqc/           -- Informes de calidad (raw y post-trim)
| trimmomatic/      -- Lecturas limpias y logs de trimming
| star_alignment/   -- BAM alineados, logs STAR, SJ tables
| samtools/         -- BAM ordenados, índices, flagstat, idxstats
| picard_markdup/    -- BAM con duplicados marcados, metricas
| featurecounts/     -- Matriz de cuentas + summary
| multiqc/          -- multiqc_report.html
|   | multiqc_data/  -- Datos brutos de MultiQC para cada módulo
| R_exploratory/     -- Boxplots y PCA (generado manualmente)
```

7. Interpretación de los Resultados

7.1. Interpretación del Control de Calidad (QC)

La evaluación del informe `multiqc_report.html` revela una mejora sustancial en la calidad de las lecturas tras el paso de Trimmomatic. La puntuación Phred media aumenta, y la proporción de lecturas con adaptadores se reduce drásticamente.

El análisis de las métricas de alineamiento (SAMtools flagstat y STAR Log.final.out) muestra un alto porcentaje de lecturas únicas mapeadas correctamente al genoma humano (GRCh38), lo que indica una buena calidad de las librerías y una adecuada elección de los parámetros de alineamiento.

Por su parte, la tabla de métricas de Picard MarkDuplicates refleja un nivel de duplicados acorde con la naturaleza del protocolo de preparación de librerías de RNA-seq. El hecho de mantener los duplicados marcados (en lugar de eliminarlos) permite que *featureCounts* evalúe de forma más honesta y robusta el peso estadístico de cada gen en el conteo final.

7.2. Interpretación del Análisis Exploratorio (R)

- **Boxplots de distribución de expresión:** La comparación visual de las distribuciones de expresión antes y después de la normalización (método TMM de *edgeR*) demuestra que los sesgos técnicos derivados de las diferencias en la profundidad de secuenciación entre las 8 librerías han sido corregidos eficientemente. La alineación de las medianas confirma la efectividad del proceso de normalización.

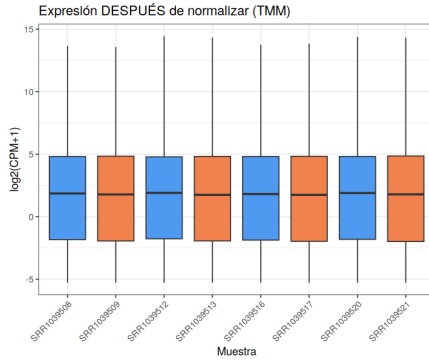


Figura 1: Boxplot de distribución de expresión normalizada (TMM)

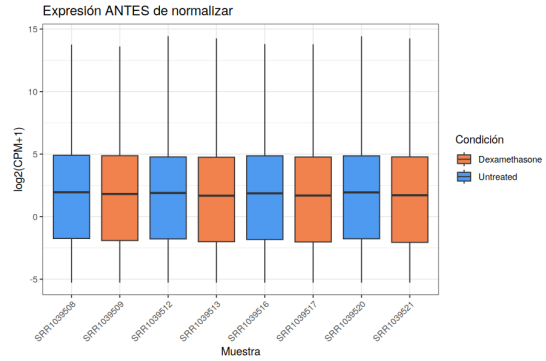


Figura 2: Boxplot de distribución de expresión cruda

- **PCA (Análisis de Componentes Principales):** La reducción de dimensionalidad no supervisada separa de manera clara las muestras en función de la condición experimental. La principal fuente de varianza (Componente Principal 1, PC1) discrimina perfectamente entre las células musculares tratadas con dexametasona y las no tratadas. Esto confirma a nivel bioinformático que el tratamiento con el fármaco ejerce un profundo y consistente impacto en el perfil transcriptómico global, siendo la condición biológica el factor principal de variación por encima del efecto individual de los donantes.

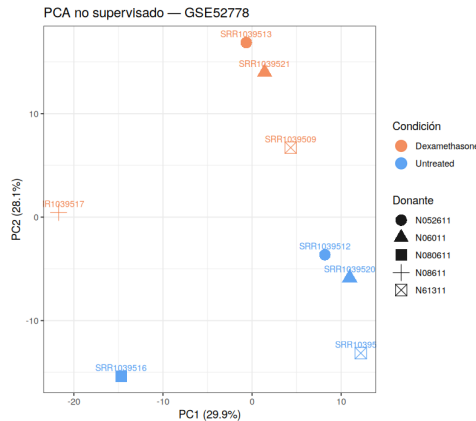


Figura 3: PCA de las muestras basado en la matriz de cuentas normalizada

8. Instrucciones de Reproducción

8.1. Requisitos Previos

Antes de ejecutar el pipeline, es necesario preparar los datos de secuenciación, el genoma de referencia y las imágenes de contenedor:

```
# 1. Datos FASTQ (requiere sra-toolkit y pigz)
chmod +x scripts/download_data.sh
./scripts/download_data.sh

# 2. Genoma de referencia (GRCh38 + GTF Ensembl 111)
chmod +x scripts/prepare_genome.sh
./scripts/prepare_genome.sh

# 3. Imágenes de contenedor (solo para HPC con Singularity)
chmod +x scripts/download_images.sh
./scripts/download_images.sh
```

Se verificará que la estructura de archivos sea la siguiente:

```
data/
| SRR1039508/...SRR1039521/
|   | SRR*_1.fastq.gz, SRR*_2.fastq.gz
| genome/
|   | GRCh38.fa, GRCh38.gtf
```

8.2. Ejecución Local (Docker)

Para ejecutar el pipeline en un entorno local con Docker:

```
nextflow run scripts/main.nf -profile docker
```

Para reanudar una ejecución interrumpida:

```
nextflow run scripts/main.nf -profile docker -resume
```

8.3. Ejecución en HPC Picasso (Singularity)

La ejecución en el clúster Picasso se realiza a través de un script de sumisión para Slurm, `scripts/script.sh`. Este script solicita los recursos necesarios para el controlador de Nextflow y luego lanza el pipeline. Es importante destacar que, aunque el controlador de Nextflow se ejecuta en el nodo asignado, cada proceso individual del pipeline (FastQC, Trimmomatic, STAR_ALIGN, etc.) se envía como un trabajo de Slurm independiente gracias al executor configurado en el perfil `picasso` de `nextflow.config`.

El comando para lanzar el pipeline es:

```
sbatch scripts/script.sh
```

Para reanudar una ejecución interrumpida, el script `script.sh` ya incluye la flag `-resume` en el comando de Nextflow, por lo que re-lanzarlo con `sbatch` reanudará el flujo desde donde se detuvo.

8.3.1. Contenido del script de lanzamiento (`script.sh`)

A continuación, se detalla el contenido del script `scripts/script.sh`, que encapsula toda la configuración necesaria para el entorno HPC:

```
#!/bin/bash
#####
# script.sh - Lanzador SBATCH para el pipeline RNA-seq en Picasso (SCBI-UMA)
```

```

#
# Uso:
# sbatch scripts/script.sh
#####

#SBATCH --job-name=rnaseq_nf
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=12
#SBATCH --mem=64G
#SBATCH --time=48:00:00
#SBATCH --output=logs/nf_%j.out
#SBATCH --error=logs/nf_%j.err
#SBATCH --partition=general

set -euo pipefail

# 1. Cargar módulos
module load singularity/3.7.2
module load nextflow/25.10.4

# 2. Verificación de entorno
command -v nextflow >/dev/null 2>&1 || { echo "ERROR: nextflow"; exit 1; }
command -v singularity >/dev/null 2>&1 || { echo "ERROR: singularity"; exit 1; }

# 3. Configurar directorio de trabajo (FSCRATCH para I/O rápido)
if [ -z "$FSCRATCH" ]; then
    export FSCRATCH="$HOME"
fi

# 4. Caché de imágenes Singularity (.sif)
# IMPORTANTE: Las imágenes deben descargarse primero en el login node
# con bash scripts/download_images.sh
export NXF_OFFLINE=true
export NXF_SINGULARITY_CACHEDIR="$HOME/UEM/nextflow_rnaseq/.singularity_cache"
export SINGULARITY_CACHEDIR="$HOME/UEM/nextflow_rnaseq/.singularity_cache"
mkdir -p "$NXF_SINGULARITY_CACHEDIR"

# 5. Directorio de trabajo de Nextflow
export NXF_WORK="$FSCRATCH/nextflow_rnaseq_work"
mkdir -p "$NXF_WORK"

# 6. Ejecutar pipeline
echo "=== Iniciando pipeline RNA-seq GSE52778 ==="
nextflow run scripts/main.nf -profile picasso -resume

```

8.4. Ejecución del Análisis en R (Workaround)

Como se ha documentado, debido a los problemas de compatibilidad de las librerías de R dentro del contenedor de Bioconductor, el análisis exploratorio debe ejecutarse manualmente utilizando la instalación nativa de R del clúster:

```
module load R/4.3.0 # o la versión disponible en el sistema
Rscript scripts/bin/rnaseq_exploratory.R
```

Los gráficos resultantes (*boxplots* y PCA) se guardarán en el directorio de salida correspondiente.